# REPASO — RESPUESTAS

Inteligencia Computacional — Unidades 2 a 7

Respuestas cortas y concisas a las 84 preguntas de repaso

# UNIDAD 2 — Perceptrón Simple

### 1. 6 características de las redes neuronales
No linealidad, paralelismo, aprendizaje, generalización, adaptabilidad, robustez.

### 2. Simplificaciones del modelo de neurona
Las entradas llegan simultáneamente, y se evalúa de forma instantánea si la suma ponderada supera el umbral u.

### 3. Modelo simplificado (3 pasos)
1) suma ponderada vs. umbral u. 2) umbral como resta. 3) umbral incorporado a la sumatoria como x0=-1, w0=u.

### 4. Ecuación formal
$v = \Sigma(i=0,n)\ xi \cdot wi \quad y = \varphi(v)$

### 5. Funciones de activación
Transforman v en la salida y. Ejemplo: función signo → cualquier negativo da -1, cualquier positivo da +1.

### 6. Sesgo / bias
Es x0=-1 con w0=u (el umbral incorporado a la sumatoria). Sirve para que la recta de separación no esté obligada a pasar por el origen.

### 7. Tipos de entrenamiento
**Corrección de error** (función signo) — pasos:

- 1) Inicialización al azar
- 2) Se muestran ejemplos con salida esperada: si es correcta, no se cambia nada (principio de mínima perturbación); si es incorrecta, se penaliza en sentido opuesto a como contribuyó
- 3) Volver al paso 2 hasta cumplir un criterio de finalización

Fórmula unificada: $w(n+1) = w(n) + (\eta/2)\cdot(yd(n)-y(n))\cdot x(n)$

**Gradiente** (activaciones derivables) — el PDF NO lo da como pasos numerados, es conceptual:

- Concepto: mover los pesos en la dirección opuesta al gradiente del error
- Ecuación básica: $w(n+1) = w(n) - \mu\nabla w\xi(w(n))$
- La velocidad de aprendizaje regula qué tan rápido te movés por la superficie de error
- Aplicación: caso simple (perceptrón, LMS: $w(n+1)=w(n)+2\mu e(n)x(n)$) / caso general (MLP: backpropagation)

# UNIDAD 3 — Redes Neuronales 2

### 8. Por qué no alcanza el perceptrón simple
El XOR no es linealmente separable (una sola recta no lo resuelve). Se soluciona combinando 3 perceptrones en 2 capas → nace el MLP.

### 9. Arquitectura del MLP
Capas de neuronas (entrada, ocultas, salida). Notación matricial: W (pesos) por capa, con superíndices indicando la capa. Bias x0=-1 en cada capa.

### 10. Activación y por qué no es signo
Sigmoide bipolar (salida -1 a +1). Se necesita que sea derivable para poder calcular el gradiente en backprop; signo no lo es.

### 11. Criterio de error
$\xi(n) = \frac{1}{2} \Sigma(j=1,M)\ ej(n)^2$ — error cuadrático instantáneo, sumado sobre las M salidas.

### 12. Propagación hacia adelante
$vj(p) = \Sigma(i=0,n)\ wij(p) \cdot yi(p-1)$  $yj(p) = \varphi(vj(p))$ — se calcula capa por capa, de la entrada hacia la salida.

### 13. Propagación hacia atrás
Se calculan los δ empezando por la capa de salida y yendo hacia atrás, capa por capa, reutilizando en cada paso los δ ya calculados en la capa siguiente.

### 14. Por qué "retropropagación"
Las capas ocultas no tienen un error propio (no hay d conocido) — su δ se arma a partir de los δ de la capa siguiente ($\Sigma \delta k \cdot wjk$), es decir, el error "viaja" hacia atrás, capa por capa.

### 15. Qué es una época
Una pasada completa por todos los patrones del conjunto de entrenamiento (propagación adelante + atrás + actualización, repetido para cada patrón).

### 16. Qué es una red de base radial y cuál es su objetivo
Red neuronal que usa funciones gaussianas (de base radial) como activación en su capa oculta. **Objetivo**: encontrar k conjuntos Cj de patrones de entrada (uno por cada neurona), de forma que cada conjunto Cj sea lo más diferente posible de los demás, y que los patrones xi dentro de un mismo Cj sean lo más parecidos entre sí. De ahí se obtiene el centroide μj de cada Cj.

### 17. Arquitectura RBF-NN
Entrada → capa radial (neuronas gaussianas, pesos fijos=1, sin bias) → capa de salida (perceptrón lineal, pesos entrenables wkj + bias w0).

### 18. Entrenamiento de RBF-NN
**Etapa 1 (no supervisada)**: k-medias arma μ y σ de cada gaussiana — por lotes (arma conjuntos, calcula centroides, reasigna, repite) u online (ajusta patrón a patrón, mueve solo el centroide ganador). **Etapa 2 (supervisada)**: LMS ajusta los pesos de la capa de salida: $wkj(n+1)=wkj(n)+\eta(dk(n)-yk(n))\varphi j(x)$.

### 19. Modelo matemático
Cada neurona radial calcula:

$$\varphi j(x) = e^{-||x-\mu j||^2 / 2\sigma j^2}$$

Y la salida final (capa lineal) combina todas las gaussianas:

$$y = \Sigma j\ wkj \cdot \varphi j(x) + w0$$

### 20. Cálculo de σ
$\sigma j = \sqrt{(1/|Cj|) \cdot \Sigma\ ||xl-\mu j||^2}$, para los $xl \in Cj$. En palabras: se toman todos los patrones que quedaron en el conjunto j, se mide qué tan lejos está cada uno de μj, y se promedian esas distancias (al cuadrado, con raíz al final) — define el ancho de la campana de Gauss: σ chico = grupo compacto (gaussiana angosta), σ grande = grupo disperso (gaussiana ancha).

### 21. Por qué "dinámica"
Porque incorpora el tiempo: la salida depende también de instantes anteriores (entradas, salidas o estados internos previos), no solo de la entrada actual.

### 22. Las 3 aproximaciones
1) Entrada expandida (x(n-1), x(n-2)...). 2) Salidas anteriores como entrada. 3) Estados internos de la capa oculta realimentados (recién acá el comportamiento es dinámico de verdad). + Combinación de las 3.

### 23. TDNN vs. RNN
**TDNN — Redes con retardo en el tiempo:**
- Tienen un comportamiento dinámico, pero las podemos entrenar con los algoritmos que ya conocemos (backprop común).

**RNN — Redes recurrentes:**
- Tienen realimentación hacia atrás (la sinapsis puede ir hacia atrás).
- **Totalmente recurrentes** (Hopfield, Boltzmann, ART): no hay capas — es una sola masa de neuronas, todas conectadas entre sí (menos consigo misma). Toda la red participa de la recurrencia a la vez.
- **Parcialmente recurrentes / PRNN** (BPTT, Elman, Jordan): sí conservan la arquitectura por capas de una red normal (entrada-oculta-salida); la recurrencia se agrega solo en un punto puntual (la capa oculta en Elman, la salida en Jordan), el resto sigue siendo feedforward.

### 24. Red de Hopfield
**Generalidades:** red dinámica totalmente recurrente que funciona como memoria asociativa — guardamos patrones (memorias fundamentales) y la red reconstruye el patrón completo aunque le mostremos una versión incompleta o con ruido (ej: reconocer un rostro aunque esté cortado).
**Arquitectura:** 1 sola capa (no hay capas ocultas, solo una masa de neuronas), todas conectadas entre sí (nunca consigo misma), misma cantidad de entradas/salidas/neuronas, pesos simétricos (wij=wji, wii=0), disparo probabilístico. Se inicializa con un estímulo de entrada; de ahí en más solo circulan las salidas de instantes anteriores (la entrada original no vuelve a entrar).
**Modelo matemático:**

$$vi(n) = \Sigma_{j \neq i}\ wij \cdot yj(n) \qquad yi(n+1) = sgn(vi(n))$$

Si vi=0, la salida se queda igual que en el instante anterior (no cambia) — hay 3 casos posibles, no solo ±1.
**Entrenamiento (almacenamiento) — Hebbiano, NO iterativo, sin error que minimizar** (guarda patrones completos, no compara contra un target):

$$wji = (1/N) \cdot \Sigma_{k=1}^{P}\ x*kj \cdot x*ki$$

Interpretación: si i y j siempre coinciden (ambas +1 o ambas -1) → peso positivo y grande; si siempre son opuestas → negativo y grande; si a veces sí y a veces no → cercano a 0.
**Prueba (recuperación) — SÍ es iterativa:**
1. Inicializar la salida en n=0 con la entrada: y(0)=x
2. Elegir al azar una neurona j, calcular su nueva salida $yj(n)=sgn(\Sigma wji \cdot yi(n-1))$
3. Repetir hasta no observar cambios en las y — cuando una pasada completa no cambia ninguna, la red convergió a una memoria fundamental

Es el proceso inverso al de las redes estáticas: ahí entrenar era largo y usar la red era instantáneo; acá guardar es directo (1 paso) y usar/recuperar es lo que itera.

### 25. Campos energéticos / valles espurios
Superficie con picos y valles que se genera durante el entrenamiento, representando qué tan estable es cada estado posible de la red. Si el entrenamiento salió bien, las memorias fundamentales quedan en los valles, y recuperar es "ir bajando" hasta caer en uno. Aparecen **valles espurios** (memorias falsas) si se superó la capacidad máxima de almacenamiento de la red ($P_{max} = N / (2 \cdot \ln(N))$), o si las memorias guardadas son muy parecidas entre sí — ahí la red puede quedar oscilando o converger a un patrón que en realidad nunca se guardó.

## 26. BPTT, Elman, Jordan

**BPTT**: expande la red recurrente en el tiempo, convirtiéndola en una red feedforward entrenable con backprop común. **Elman**: realimenta la capa oculta hacia sí misma. **Jordan**: realimenta la salida.

# UNIDAD 4 — Aprendizaje Automático

### 27. Superficies de error con muchos mínimos
Riesgo de quedar atrapado en un mínimo local en vez del global. Se aborda con múltiples inicializaciones al azar, o con métodos poblacionales (evolutivos) que buscan en varios puntos del espacio a la vez.

### 28. Sobre-entrenamiento
La red memoriza detalles del training set en vez de generalizar (ocurre con muchas épocas o muchos parámetros libres). Se evita con un subconjunto de **monitoreo**: se corta el entrenamiento cuando su error deja de bajar y empieza a subir (early stopping).

### 29. Validación cruzada
**Leave-k-out / k-fold**: N = total datos / k dejado afuera; se repite, y se promedian los N errores (+ varianza). **Leave-1-out**: caso k=1, exhaustivo pero costoso. **Solapar particiones**: alternativa más liviana, pero pierde independencia estadística entre experimentos.

### 30. kNN
Busca los k vecinos más cercanos (distancia) y vota por mayoría. "No aprende": no hay parámetros que se ajusten, todo se calcula en el momento. **Ventajas**: simple, sin entrenamiento. **Desventajas**: elegir k, costoso (compara contra todos los datos), sensible a outliers y desbalance.

### 31. Árboles de decisión
Se elige la variable/corte que más reduce la impureza (Gini o Entropía), vía ganancia de información = impureza del padre − impureza de los hijos. **Ventajas**: interpretable, maneja datos mixtos, bajo costo. **Desventajas**: sensible al ruido, fronteras ortogonales, se sobreajusta fácil.

### 32. Naive Bayes
"Ingenuo" porque asume que todas las variables son independientes entre sí dada la clase → la probabilidad conjunta se separa en una productoria simple $P(x1|w)\cdot P(x2|w)...$ **Ventajas**: rápido, mezcla datos. **Desventajas**: la independencia rara vez es real; probabilidad 0 si una combinación nunca se vio.

### 33. SVM
Busca el hiperplano que maximiza el margen entre clases. Vectores de soporte = los puntos más cercanos al margen. Truco del kernel: proyecta a más dimensiones para separar lo que no es separable linealmente. **Ventajas**: buena generalización, altas dimensiones. **Desventajas**: sensible al kernel elegido, datasets grandes, clases solapadas.

### 34. Cuadro comparativo
kNN (simple, no aprende, costoso) — Árboles (interpretable, fronteras ortogonales, se sobreajusta) — Naive Bayes (rápido, independencia poco realista) — SVM (buen margen, sensible al kernel).

### 35. Bagging
Bootstrap = muestreo con reposición (genera B muestras distintas del mismo dataset). Se entrena un modelo por muestra, y se combina por votación (clasificación) o promedio (regresión).

### 36. Stacking
Nivel 1: varios modelos simples entrenados con los datos originales. Nivel 2: un meta-modelo que recibe las salidas de nivel 1 como entrada, y aprende a combinarlas — se entrenan por separado.

### 37. Boosting por muestreo vs. AdaBoost
**Boosting por muestreo**: C1 (bootstrap normal), C2 (mitad bien / mitad mal clasificados por C1), C3 (donde C1 y C2 difieren). **AdaBoost**: cada patrón tiene un peso — sube si falla, baja si acierta; voto final pesado según el desempeño de cada clasificador.

### 38. Gradient Boosting
Cada modelo nuevo predice el residuo/error acumulado de los anteriores (no los datos originales otra vez). Predicción final = suma de todas las salidas. Ej: casa real=120.000 → Tree1=100.000 (resid.20.000) → Tree2 predice 15.000 → Tree3 predice 4.000 → total=119.000.

### 39. Ventajas/desventajas de ensambles
**Ventajas**: poco sobreajuste, manejan desbalance, combinan tipos de datos, buenos con datos escasos, multiclase, robustos. **Desventajas**: entrenamiento costoso, poco interpretables, dependen de la diversidad de los modelos base.

### 40-41. SOM: qué es y arquitectura
Auto-organización = interacciones locales → orden global. Arquitectura: 1 sola capa organizada en un plano, conexión todos-contra-todos (cada entrada a cada neurona).

### 42. Entrenamiento del SOM
No supervisado, competitivo. Neurona ganadora = menor distancia entre su vector de pesos y la entrada. Algoritmo: 1) inicialización (azar o = un patrón), 2) selección del ganador (argmin distancia), 3) adaptación: solo se actualiza la ganadora y su entorno $\Lambda g$; el resto queda igual.

### 43. Ag y η variables/decrecientes — 3 etapas
**Ordenamiento global**: $\Lambda g$ grande (~medio mapa), η alto (0.9-0.7), 500-1000 épocas. **Transición**: $\Lambda g$ baja a 1, η baja a 0.1, ~1000 épocas. **Ajuste fino**: $\Lambda g=0$ (solo ganadora), η bajo constante, hasta converger (~3000 épocas).

### 44. Mapas topológicos: con y sin vecindad
Sin vecindad: cada neurona agrupa bien pero de forma aislada (como k-medias, sin orden). Con vecindad: además de agrupar, se logra ordenamiento topológico — neuronas cercanas en el mapa representan grupos parecidos.

### 45. LVQ vs. SOM
LVQ es supervisado y sin vecindad. Si el prototipo ganador acierta la clase → se acerca al patrón; si falla → se aleja. Es como un "SOM lineal, sin entorno y supervisado".

### 46. Clustering
Agrupar datos sin etiquetas, según una noción de distancia entre patrones.

### 47. Clustering jerárquico
Aglomerativo: cada punto arranca como su propio cluster y se van uniendo los más cercanos. Divisivo: arranca todo junto y se va dividiendo. Se representa con un dendrograma (la altura de unión = distancia).

### 48. Métricas de clustering
**Internas**: cohesión (compacidad), separación (distancia entre clusters), Davies-Bouldin (combina ambas). **Externas**: Fowlkes-Mallows, Rand Index (comparan contra otra partición conocida).

### 49. Número óptimo de clusters (k)
Se corre el algoritmo con distintos k, se grafica la métrica resultante, y se busca un "codo" o pico en esa curva.

# UNIDAD 5 — Inteligencia Colectiva 1

### 50. Autómatas celulares ⚠
Autómata: A=<X,Y,E,D> (entradas, salidas, estados, reglas de transición, det. o probabilísticas), representable en grafos. Celular: R=<A,T,C> (autómatas, topología, conectividad/vecindad: Von Neumann, Moore). Conway: vive con 2-3 vecinos, nace con 3, muere si <2 o >3.

### 51. Agentes ⚠
Un agente es un sistema situado en un ambiente, capaz de realizar acciones automáticas para cumplir sus objetivos de diseño. Inteligente = proactivo (actúa sin que se lo pidan) + reactivo (responde a cambios del ambiente).

### 52. Sistemas multiagente ⚠
Varios agentes que interactúan y se coordinan entre sí para resolver un problema conjunto, en vez de depender de un único agente centralizado.

### 53. Características generales de IC
Auto-organización (interacciones locales → orden global), estigmergia (comunicación indirecta vía el entorno), comportamiento emergente, componentes estocásticas, bio-inspiración, colaboración vs. competencia.

### 54. Ejemplo de la jirafa (Darwin)
Si hay variabilidad en el largo del cuello, las jirafas de cuello corto sobreviven/procrean menos → la próxima generación tiene menos cuello corto. Es variación + selección natural — la semilla de pensar la evolución como un algoritmo de optimización.

### 55. La evolución como algoritmo
Poblaciones (no un individuo aislado). Mejor adaptado ≠ "súper individuo" único. Aleatoriedad: el mejor adaptado no tiene asegurada la supervivencia. Diversidad: necesaria, si no la población no puede adaptarse a cambios de contexto.

### 56. Elementos de un algoritmo evolutivo (5)
1) Representación de individuos. 2) Función de aptitud (fitness). 3) Estrategia de selección. 4) Operadores de variación. 5) Reproducción y reemplazo generacional.

### 57. Genotipo y Fenotipo
Cromosomas con genes, que toman valores (alelos). Genotipo = información en bits (0/1). Fenotipo = la expresión/valor real del problema. Codificación = proceso de pasar de genotipo a fenotipo para poder evaluar el fitness.

### 58. Función de aptitud
Mide con un número qué tan buena es una solución. Características deseables: monotonicidad, continuidad en el espacio (robustez/inmunidad al ruido), suavidad regulable.

### 59. Selección: ruleta y ventanas
**Ruleta**: probabilidad de cada individuo ∝ su fitness; se gira tantas veces como padres se necesiten. **Ventanas**: se saca al azar de una ventana que arranca grande (desde el mejor) y se va achicando. Ambas terminan cuando ya se eligió la cantidad de individuos necesaria.

### 60. Operadores de variación
**Mutación**: cambia un bit al azar → aumenta diversidad, explora regiones lejanas (posible mínimo global). **Cruza**: intercambia material genético entre 2 padres desde un punto de corte → explota/refina una zona ya buena.

### 61. Exceso de mutación / cruza
Demasiada mutación → casi aleatorio, no converge bien (pierde la capacidad de afinar). Demasiada cruza (poca mutación) → converge rápido pero a un mínimo local, se pierde diversidad.

### 62. Reproducción y reemplazo
Total (los hijos reemplazan a toda la población). Con brecha generacional (quedan algunos padres + hijos nuevos). Elitismo (el mejor individuo pasa directo a la próxima generación, para no perderlo).

### 63. Estrategias de Evolución (ES)
Surgen de hacer variables los parámetros (prob. mutación/cruza, tamaño población, brecha, elitismo). Representación fenotípica (real). Mutación se guarda solo si no empeora. Reproducción determinística: (μ+λ)-ES (próxima gen = mejores de {μ∪λ}) o (μ,λ)-ES (próxima gen = mejores de λ, con λ>μ).

### 64. Representación: Genético vs. Evolutivo
**Genético (binaria)**: muchos genes, pocos alelos; convergencia asegurada (teorema de esquemas); epistasis (1 gen malo invalida el cromosoma).
**Evolutivo (real/fenotípica)**: pocos genes, muchos alelos; convergencia depende de los operadores.

### 65. Programación Genética: representación
Genera automáticamente programas, representados como árboles. Elementos: variables/constantes, operadores aritméticos/lógicos, funciones, condicionales, bucles, recursión.

### 66. PG: cruza y mutación
**Cruza**: numerar nodos, elegir puntos de corte, intercambiar ramas completas (mantiene la función central de cada árbol). **Mutación**: elegir una rama, generar un árbol al azar, reemplazar esa rama (necesita una función generadora de árboles al azar).

### 67. Manejo de restricciones (5 formas)
Redefinir la representación (fenotipos siempre válidos), rechazo de individuos inválidos, reparación del material genético, modificar los operadores de variación, penalización en la función de aptitud.

# UNIDAD 6 — Inteligencia Colectiva 2

### 68. Estigmergia y puente binario
Estigmergia = comunicación indirecta modificando el entorno (feromonas). En el puente binario, el camino corto se recorre y se vuelve antes → deposita feromona antes → más hormigas lo eligen (con más probabilidad, no seguro) → retroalimentación positiva hacia el camino corto.

### 69. Definiciones de ACO
Grafo G=(V,E). σij = feromona en la conexión i-j. Ni = nodos disponibles desde i. p^k(t) = camino recorrido por la hormiga k.

### 70. sACO
p_ij = σij^α / Σσiu^α (solo mira feromona). Los ciclos se detectan y eliminan recién DESPUÉS de construir el camino completo.

### 71. AS vs. sACO
Agrega heurística η=1/dij (elevada a β, junto con la feromona). Usa lista tabú Ni^k (evita ciclos desde el principio, no después). 3 formas de depositar feromona: Global (Q/f del camino completo), Uniforme (Q fijo), Local (Q/dij del tramo).

### 72. α y ρ
**α**: peso de la feromona en la decisión — alto = explota (converge rápido, riesgo de óptimo falso); bajo = explora más (converge lento); α=0 = feromona no influye, no converge. **ρ**: evaporación — alto = "olvida" rápido lo malo (ayuda a corregir un mal arranque); bajo = lo acumulado queda grabado con fuerza.

### 73. PSO: definiciones
xk(t) = posición de la partícula k. vk(t) = velocidad. yk = mejor posición personal (menor error histórico). ŷ = mejor posición del enjambre (global o local según topología).

### 74. Fórmulas de PSO
xk(t+1) = xk(t) + vk(t+1)
vki(t+1) = vki(t) + c1r1i(yki-xki(t)) + c2r2i(ŷi-xki(t))

### 75. gBest vs. lBest
**gBest** (topología estrella): ŷ único, compartido por todo el enjambre — converge rápido, más riesgo de quedar en un óptimo falso. **lBest** (topología anillo): ŷk propio por vecindad — converge más lento, pero más resistente (la buena info se propaga de a poco, de vecino en vecino).

### 76. Ventajas y desventajas gBest vs. lBest
**gBest**: converge más rápido (info llega a todos de inmediato), pero mayor riesgo de convergencia prematura a un óptimo falso. **lBest**: converge más lento, pero mantiene más diversidad y es más resistente a quedarse atrapado en un óptimo falso.

# UNIDAD 7 — Lógica Borrosa ⚠

**77. Incerteza vs. aleatoriedad**
Lógica borrosa modela la incerteza sobre UN objeto (ej: qué tan cerca está algo que se acerca). Aleatoriedad es estadística sobre MUCHOS objetos (ej: qué número saldrá al tirar un dado).

**78. Conjunto binario vs. borroso**
Binario: pertenece (1) o no (0). Borroso: función de membresía con grados intermedios entre 0 y 1 — pertenece "un poco" o "bastante".

**79. Subconjunto borroso**
Conjunto de pares (elemento, grado de membresía). Inclusión, igualdad y complemento se definen comparando esos grados entre conjuntos.

**80. Operadores y geometría borrosa**
Los conjuntos borrosos se pueden representar como puntos dentro de un hipercubo (no solo en los vértices, como el caso binario).

**81. Entropía borrosa**
Mide qué tan "ambiguo"/borroso es un conjunto. Es máxima en el "conjunto medio" (todos los elementos con membresía 0.5 — máxima incertidumbre).

**82. FAM (Fuzzy Associative Memory)**
Mapea un conjunto borroso de entrada a uno de salida mediante reglas SI-ENTONCES. Regla simple (local FAM) = una sola regla. Composición de reglas (global FAM) = combina varias reglas activadas a la vez.

**83. Fuzzy Hebb FAMs**
2 formas de codificar la regla (al estilo Hebbiano): correlación-mínimo y correlación-producto.

**84. Composición: superposición vs. SAM**
Superponer matrices es la forma simple. El SAM (Modelo Aditivo Estándar) pondera cada regla con su propio factor de activación — es mejor porque respeta cuánto se activó cada regla individualmente, en vez de tratarlas todas igual.

**85. Defuzzyfication**
Convierte el conjunto borroso de salida en un número real. **Maximum-defuzzyfication**: toma el valor con la membresía más alta. **Centroid-defuzzyfication**: promedio de los valores, ponderado por sus membresías.

**86. Extensiones de FAM**
Conjuntos de pertenencia continuos (no solo discretos), representación de dos antecedentes por consecuente, composición de dos antecedentes.